

A SOLID-BASED FLEET MANAGEMENT ARCHITECTURE FOR SECURE IOT DEVICE AUTHORIZATION

George Popescu-Craiova

King's College London

30 April 2026

WHY SOLID FOR IOT FLEETS?

- ▶ IoT platforms typically couple *device identity*, *authorisation*, and *data storage* into a single proprietary cloud silo.
- ▶ Result: vendor lock-in, weak revocation guarantees, and no clear data ownership.
- ▶ **Goal** : a fleet management architecture that is revocable, federated, and scalable — without sacrificing data sovereignty.
- ▶ **Approach**: Solid pods as sovereign data containers + OAuth 2.0 client credentials per device + a separate control-plane backend.

TWO DEPLOYMENT CONTEXTS

Personal

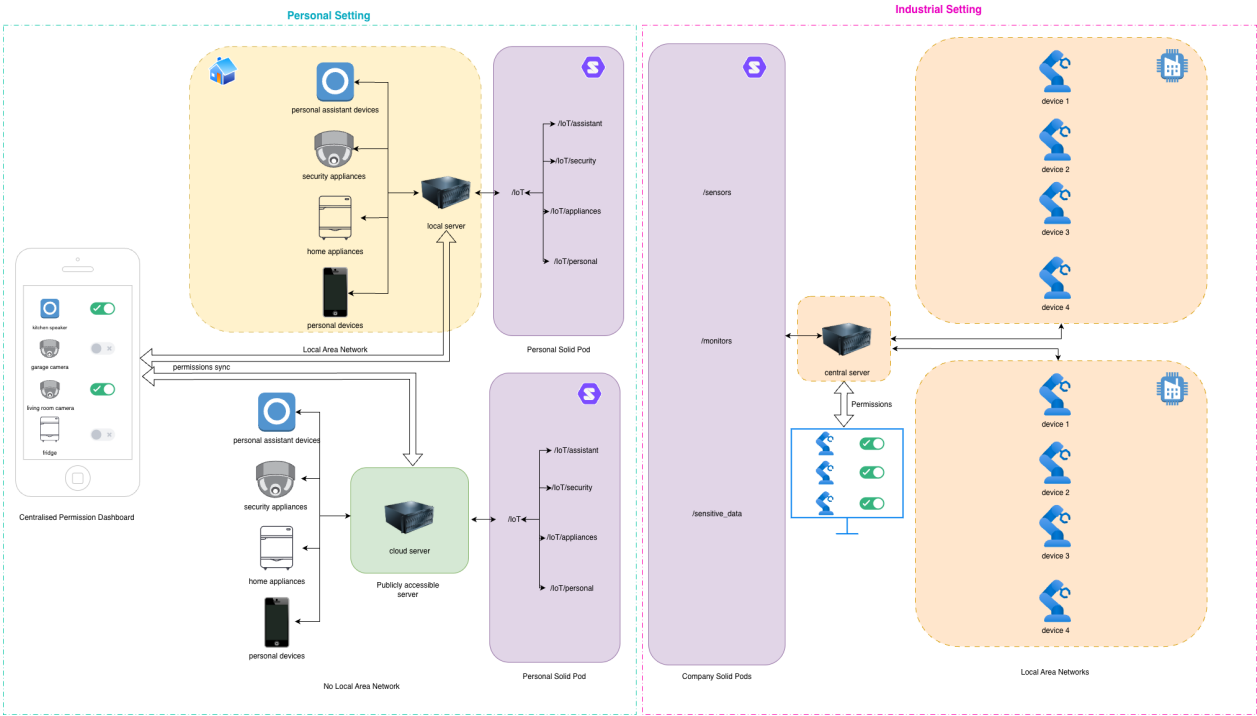
- ▶ Cameras, appliances, assistants, wearables
- ▶ All write to one personal pod
- ▶ Owner administers permissions from a mobile app

Industrial

- ▶ Federated sensors, robots, monitoring devices
- ▶ Distributed across sites, organisational pods
- ▶ Centralised authorisation, decentralised storage

Same architecture, different scale.

OVERALL ARCHITECTURE



Personal (left) and industrial (right) deployments. Backend acts as a control-plane authority only — it does not proxy device data traffic.

THREE COMPONENTS, TWO PLANES

iOS app

- ▶ Human admin interface
- ▶ Logs in via backend
- ▶ Manages fleet state

Backend

- ▶ Control-plane only
- ▶ Issues / revokes credentials
- ▶ Holds CSS account token
- ▶ Never sees device data

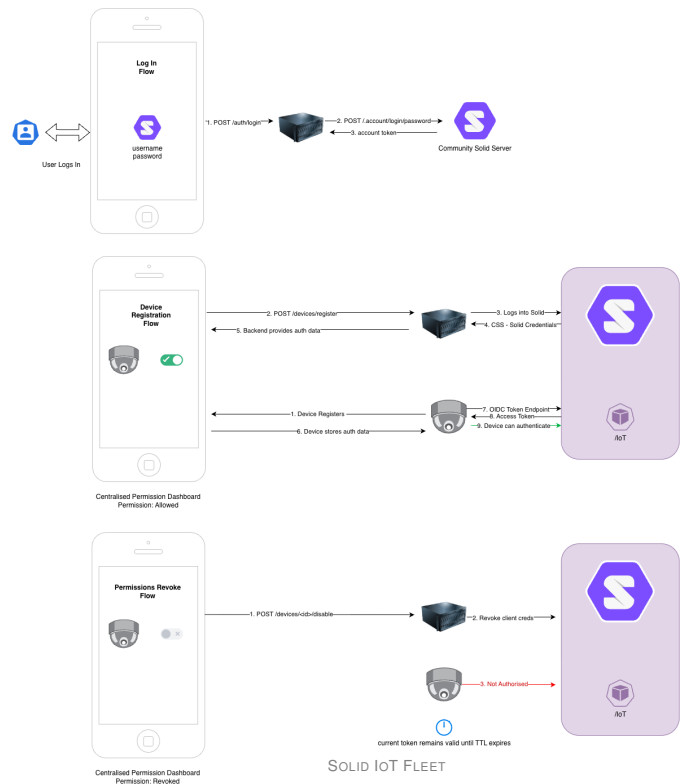
IoT devices

- ▶ Non-interactive OIDC auth
- ▶ Write directly to pod
- ▶ One credential each

Control plane = backend ↔ CSS Account API.

Data plane = device ↔ CSS pod (direct, no backend in path).

AUTHENTICATION & LIFECYCLE FLOWS



USER LOGIN: CONTROL-PLANE AUTHENTICATION

- ▶ iOS app does **not** authenticate directly with Solid.
- ▶ It logs in to the backend, which then performs a CSS Account API login: `POST /.account/login/password`
- ▶ CSS returns an account-scoped token — retained **server-side only** .
- ▶ Backend uses this token to create / revoke device credentials bound to the user's WebID.

Clean separation : human auth (interactive) vs. machine auth (non-interactive). Devices keep working even when no user is logged in.

DEVICE REGISTRATION

1. New device exposes a pairing endpoint; iOS app retrieves metadata.
2. App calls `POST /devices/register` on the backend.
3. Backend invokes CSS Account API $\Rightarrow$ creates an OAuth 2.0 client credential bound to the owner's WebID.
4. CSS returns: `client_id`, `client_secret`, and a [credential resource URL](#) (used for later revocation).
5. Credentials transmitted to the device during provisioning; secret stored encrypted in backend DB.

Device persists a local `device.json` manifest: secret, issuer URL, authorised container path.

CONTAINER-LEVEL WAC GRANULARITY

Solid enforces authorisation [at the container level](#) via Web Access Control:

```
/iot/security/camera-1/  
/iot/security/camera-2/  
/iot/appliances/fridge/  
/iot/assistant/speaker/
```

- ▶ Each container's ACL specifies which WebIDs / client credentials can Read, Write, or Append.
- ▶ A camera **cannot** write to the assistant container.
- ▶ A fridge **cannot** modify camera footage.
- ▶ Industrial: a robotic arm appending telemetry is structurally barred from other containers / pods.

[Bounded blast radius](#) : compromised credential leaks one container, not the fleet.

DATA-PLANE AUTHENTICATION (OIDC)

Devices authenticate **directly** with CSS via the OAuth 2.0 client credentials grant (RFC 6749 §4.4):

```
POST /.oidc/token
grant_type=client_credentials
```

- ▶ CSS validates `client_id` + `secret`, returns a short-lived bearer access token (RFC 6750).
- ▶ Backend is **never** in the data path; it cannot forge tokens.
- ▶ CSS is the **sole enforcement authority** on the data plane.
- ▶ Network-path independent: home WiFi, industrial LAN, cellular — anything that reaches the CSS issuer.

DISABLE FLOW: SERVER-SIDE REVOCATION

When the user disables a device:

1. iOS app: `POST /devices/{id}/disable` → backend.
2. Backend calls CSS Account API ⇒ `deletes the credential resource` .
3. CSS removes the credential from its registry.

Result:

- ▶ Any new token request fails with `invalid_client`.
- ▶ Existing access tokens remain valid `only until natural expiry` .
- ▶ Revocation latency $\leq$ access-token TTL.
- ▶ Local credentials on the device become `cryptographically inert` — the issuer no longer recognises them.

RE-ENABLE: CREDENTIAL ROTATION, NOT REACTIVATION

- ▶ Re-enabling does **not** restore the old credentials.
- ▶ Backend issues an **entirely new** client credential.
- ▶ Optional: device performs a refresh handshake using a long-lived bootstrap secret; backend pushes new credentials on validation.

Forward secrecy property

An attacker holding a stale (revoked) secret derives *no advantage* from a subsequent re-enablement. Each lifecycle event creates a fresh credential.

SECURITY GUARANTEES, SUMMARISED

Per-device identity One OAuth 2.0 client credential per device $\Rightarrow$ fine-grained, immediate revocation.

Server-side enforcement Revocation happens at CSS, not at the device $\Rightarrow$ inert credentials post-disable.

Namespace containment Container-level WAC bounds blast radius.

Plane separation Backend never sees data; CSS never sees control-plane secrets.

Decentralisation Centralised lifecycle control *without* centralised storage.

CLOSING

Solid + CSS as a **decentralised authorisation and storage layer** for IoT fleets.

Revocable. Federated. Scalable. Data-sovereign.

Future work: formal threat modelling · JWT client assertions (RFC 7523) · automated WAC policy synthesis.